CheckD 四人团队开发计划初学者执行指南

> 版本：2026-10-29。供队长、同学 A、B、C 共同执行。原计划日期为 2026-10-28 至 11-08；日期是原定目标，不表示任务已完成。10-28 已过去，先由负责人核对实际产物与证据，再在看板上标注“完成 / 进行中 / 阻塞”。

1 先看懂最终要做成什么

原计划要求： 11 月 8 日内部交付一个在腾讯官方 WorkBuddy 上可由其他账号访问的 Agent。它听懂学生的安排请求，调用 CheckD 的真实工具读取任务和课程，给出可行计划；用户看过并确认后才写入，随后再读一次数据证明写入成功。至少演示“查明天”“拆报告并排程”“插入会议后重排”三种情形。

通俗解释： WorkBuddy 是学生说话的入口，CheckD 是保存事实的账本，调度器只负责算建议。Agent 自己说“已保存”不算完成；必须有工具调用记录和写入后的回读结果。比赛要求一个真实可访问的 WorkBuddy Agent 链接。

实际验收标准： 其他账号能打开链接并触发工具；三条场景各连续成功三次；固定课程不被移动；未确认时没有正式写入；失败时显示失败；至少十条边界用例有真实记录。这些是团队内部标准，不是已确认的官方评分阈值。

待确认事项： WorkBuddy 当前账号的分享权限、可用工具类型、远程部署方式，线上 Supabase 迁移状态和云端数据问题都未实证。赛事说明是征求意见稿，正式截止时间需队长向组织方核实。不能把这些写成“已解决”。

1.1 项目里什么已经存在

- Flutter 是跨平台界面框架，Dart 是本项目主要开发语言。iOS/macOS 有少量 Swift 平台代码，Task/Calendar/Timer 的主业务代码在 lib/。
- 本地任务模型在 lib/features/tasks/domain/task_models.dart；TaskRepository 在 lib/features/tasks/data/task_repository.dart。现有 getTask、watchTasksForDate、saveOneTimeReminder、scheduleOneTimeTask 以及计时方法是应用内部方法，不是现成的远程 Agent API。
- 日历在 lib/features/calendar/domain/calendar_occurrence_builder.dart 把 Course、Task、单日例外合成 CalendarOccurrence；单日例外在 lib/features/schedule/data/day_schedule_repository.dart。仓库没有独立持久化的 ScheduleBlock 表。
- Drift 本地表在 lib/core/database/app_database.dart；同步入口在 lib/core/sync/sync_providers.dart，上传、下载、队列、账户协调和首次同步分别在同目录的 sync_upload_service.dart、sync_download_service.dart、sync_queue_service.dart、sync_account_service.dart、sync_bootstrap_service.dart；云端表与 RLS 规则在 supabase/migrations/。
- 2026-10-28 的六组相关基线测试曾有 93 项通过。它只证明当时的相关本地测试；不证明今天的线上同步、WorkBuddy 或新接口可用。

1.2 每个人都必须守住的边界

Course 不是 Task；周期任务与单次事项分别建模；TimerSession 是实际计时，提案时间块是未来安排；计划日期、计划时间、实际时间和手动完成互不替代。计时到达目标不自动完成。不同任务可同时计时，同一任务最多一条未完成计时；全局专注统计对重叠时段只计一次。调整某一天的课程或周期事项，要用单日例外，不能改后续周期规则。固定时间事项的到点提醒不能被提前提醒取代。不得在 Agent MVP 中改计时表、完成记录或统计口径。

所有外部写入都要先鉴权、再检查用户批准和数据版本，最后回读；不允许把用户 ID 从请求正文直接当成身份，不允许提交密钥、真实个人课表或未经授权的数据。若某项能力不可用，记录 BLOCKED，给出可复现的错误和下一步，不用静态截图假装成功。

2 四个人怎样接力

队长冻结 v1 数据契约
├─ A 用 Mock 先验证 WorkBuddy 分享与工具调用
├─ B 用同一契约和假数据写纯调度器
└─ C 用假数据准备场景和验收表
↓
队长通过云端数据关口并提供真实只读接口
↓
A 真实读取 → B 给调度提案 → A 展示差异
↓
用户明确批准 → 队长服务校验并写入 → 再读核验
↓
C 端到端测试和证据 → 11-06 冻结 → 11-07 彩排 → 11-08 内部交付

为什么先冻结 v1 契约？ 契约就是四人共用的数据字典：字段名、日期/时区、错误格式、工具名和提案状态。若 A 叫 due_at、B 叫 deadline，联调时会失败。队长把 docs/contracts/agent-api-v1.md 和脱敏 JSON 样例审定后，其他人按同一份开发；修改它要先提 Issue。

为什么 A、B、C 能并行？ Mock 是按契约写的假响应，A 可先验证 WorkBuddy 是否真的调用工具。B 的 generateSchedule(ScheduleInput input) 只接受输入并返回提案，不碰数据库。C 可先写虚拟课程、任务和预期结果。三人都不必等待真实云端接口；但真正的“已保存”证据必须等 M0 的真实写入与回读。

关键路径与阻塞点： 10-29 分享链接与 v1 契约 → 云端数据关口和 10-30 真实只读 → 11-02 无冲突提案 → 审批/幂等/版本校验 → 11-04 写入回读 → 11-06 冻结。若分享不可达，暂停高级排程，先修比赛入口；若云端数据不可信，只保留 Mock/只读演示并标明；若原子写入无法证明，不宣称写入成功。

2.1 云端数据正式关口

原计划要求： M0 在做真实只读与写入前核对 Supabase 与本地 Drift 的映射、用户权限及同步收敛。用户补充说明“云端数据尚有未完全解决的问题”。

通俗解释： 手机看到的一条任务和云端保存的一条任务必须对应同一用户、同一 ID、合理版本及计划时间。若云端漏掉任务、重复任务、旧版本覆盖新版本，Agent 读到或写入的就可能是错误账本。

队长按顺序检查： ①使用脱敏测试账号记录本地和云端任务/课程/单日例外/计时数量及 ID；②查看同步页状态、待处理/失败队列与错误码，不复制真实密钥；③核对 SyncDownloadExecutor.reconcile()、SyncUploadExecutor.flush()、SyncBootstrapService.bootstrap() 和账户切换行为；④检查线上实际迁移版本与 supabase/migrations/ 一致性、RLS 是否拒绝跨用户读取；⑤按“本地新建→上传→另一设备下载→修改→再次同步”走一次最小可复现流程；⑥把预期、实际、时间、测试账号、错误码、影响表、复现步骤写进 M0 的问题记录，再根据证据确定修复范围。

通过才允许真实写入： 测试账号的任务/课程/例外可稳定回读，跨用户访问被拒绝，版本冲突有明确结果，写入不会被下一轮同步回滚或重复。当前状态：待验证，根因未定。本指南不批准直接运行线上迁移或改同步代码；修复须由 M0 另开独立 PR、测试后人工合并。

3 每人的职责和首个 PR

3.1 队长 M0 数据接口、权限与集成

目的： 给 WorkBuddy 一个受控的数据门，而非让 Agent 直接操作数据库。先能安全读，再能生成待批准提案，最后在版本不变且用户批准后写入并回读。

先读的真实代码： lib/features/tasks/domain/task_models.dart、lib/features/tasks/data/task_repository.dart、lib/features/calendar/domain/calendar_occurrence_builder.dart、lib/features/schedule/domain/day_schedule_models.dart、lib/features/schedule/data/day_schedule_repository.dart、lib/features/courses/data/course_repository.dart、lib/core/database/app_database.dart、lib/core/sync/ 和 supabase/migrations/。特别关注 TaskTimeSemantics.validate、DayScheduleRepository.saveDailyOverride 与 SyncRemoteReader.readTable 的真实签名。

第一交付 M0-1（原定 10-29）： 新建 docs/contracts/agent-api-v1.md 和 docs/contracts/agent-api-v1.examples.json。输入是现有模型、A 的平台工具能力、B/C 所需字段；输出是 list_tasks、get_calendar_events、generate_schedule、validate_schedule、apply_approved_changes 的 v1 JSON 字段与例子。工具名为拟建协议，不是现成函数。区分日期型截止与带时区精确时间；snapshot 包含任务、Course、单日例外、版本；proposal 包含 proposal_id、base_version、changes、unplanned_tasks、explanations。单次事项现有一个具体计划时间，不能在契约中假称已有多个持久时间块。运行 python3 -m json.tool docs/contracts/agent-api-v1.examples.json >/dev/null，退出码 0，再让 A/B/C 逐字段确认。

第二交付 M0-2（原定 10-30）： 前置是 A 证明 WorkBuddy 工具可达、云端数据关口给出可用读路径。实现真实只读任务/日历映射；身份只能来自服务端验证的 token。先写无 token、过期 token、跨用户、空列表和本地/云端一致性测试，再实现、运行新测试以及 flutter test test/task_repository_test.dart test/calendar_occurrence_builder_test.dart test/sync_download_service_test.dart。服务文件路径、部署命令和密钥注入方式 待验证，不能虚构 server/checkd-api/ 已存在。

第三交付 M0-3（原定 11-04）： 前置是 B 合法提案、A 人工确认流和云端写入关口通过。先测试“未批准零写入、重复批准只执行一次、跨用户拒绝、版本漂移拒绝、固定课程不可移动、回读不一致不报成功”，再实现 validate_schedule 与 apply_approved_changes。审批凭据由服务端核验，幂等键阻止重复执行，版本检查阻止旧方案覆盖新数据。写入只涉及获批准的单次事项计划或单日例外，不改 TimerSession/Completion/周期规则。若需迁移，独立 PR，先验 RLS/事务/线上版本。回归命令见第 8 节。

M0 完成标准： 其他人能按 v1 Mock 开发；真实读只返回授权用户数据；写入要有批准、幂等、版本及回读证据；旧计时和日历测试通过。队长负责最终 PR 审查与人工合并。

3.2 同学 A M1 WorkBuddy Agent

目的： 证明比赛指定的平台确实能把自然语言变为工具调用，最后能让其他账号使用，而不只是显示一段会说话的提示词。

第一交付 M1-1（原定 10-29）： 在腾讯官方 WorkBuddy 创建最小 Agent；先用另一账号/独立浏览器打开分享链接，记录 URL、分享范围、时间、失败码和截图；再按 M0 草案用 Mock 配只读工具，输入“明天有哪些任务”，比对工具调用 ID、返回 JSON 和 Agent 回答。文档新建 docs/workbuddy/access-check.md、docs/workbuddy/tool-mapping.md。若本地 MCP 不可达，验证平台支持的远程 HTTPS 或托管方式并回告 M0；平台能力未实证前标 待验证。通过标准是非本人账号打开链接并触发真实的 Mock 工具调用。

第二交付 M1-2（原定 11-03）： 等 M0-2 真实只读与 B 的基础调度接口后，把 Mock 替换为真实工具；先测三条脱敏提示词的读取，再加“拆报告→生成提案→解释计划与未安排量”，仍不写库。保存 docs/workbuddy/workflow-v1.md、docs/workbuddy/prompts-and-failures.md，记录调用日志中的用户/任务 ID 与回答一致。模拟 401、超时、空结果、含恶意指令的知识文本；回答要说“未执行”。

第三交付 M1-3（原定 11-06）： 等 M0-3 可安全写入后，在 WorkBuddy 显示具体变化，取得用户明确确认，再调用 apply_approved_changes，最后调用只读工具回读；保存 docs/workbuddy/verification-log.md。未确认、拒绝、重复点击、工具失败、回读不一致时都不能显示成功。异账号异网络跑三条场景各三次并留去敏日志。

为什么顺序不能跳： 分享失败意味着评委进不来；Mock 工具失败意味着平台接不住工具；真实读取失败意味着提案基于假事实；未经批准写入违反约束；不回读就无法证明数据库真的变化。

3.3 同学 B M2 智能调度

目的： 给出能解释、可重复、无硬冲突的计划建议，突发会议后只改必要的柔性任务。调度器只计算，不直接碰 Drift 或 Supabase。

第一交付 M2-1（原定 11-02）： 待 M0 v1 字段冻结，在拟新建 lib/features/agent_scheduler/domain/schedule_contract.dart 定义 ScheduleInput、ScheduleProposal、仅作提案的 ProposedBlock；在 schedule_engine.dart 拟实现 ScheduleProposal generateSchedule(ScheduleInput input)。输入含时区、可用窗口、需求分钟、固定课程/会议、现有柔性安排、baseVersion；输出含变化、未安排量、原因。先在 test/agent_scheduler_test.dart 写五组失败测试：避让课程会议、守截止、时窗不足、固定项重叠不非法移动、相同输入稳定；确认失败后实现最小规则，再测通过。日期型截止不强造 23:59。

第二交付 M2-2（原定 11-04）： 在同目录加 ScheduleProposal repairSchedule(ScheduleInput input, ScheduleProposal previous)，先测插入会议只移动冲突柔性块、无解解释、跨日不越界、重复输入结果稳定。算法先保留不受外部依赖影响的确定性规则。Horolog 只是可选参考；若要引入，先把版本、LICENSE、部署义务、对照结果写入 docs/scheduler/algorithm-and-license.md，不要求完整移植。测试命令：flutter test test/agent_scheduler_test.dart test/calendar_occurrence_builder_test.dart test/task_time_semantics_test.dart。

完成标准： 至少五组构造用例无固定项冲突；不能排下的分钟数和原因清楚；重排尽量少改；调度模块零数据库写入。M0 负责把真实 CheckD 数据映射进 B 的 DTO。

3.4 同学 C M3 场景、验收与比赛材料

目的： 让团队知道“怎样算成功”，留下评委可核对的真实证据，同时把复杂结果解释给学生。

开发前可做的 M3-1（原定 10-29）： 在拟新建 docs/contest/acceptance-cases.md 写三组虚拟学生数据：查明天、拆报告排程、临时会议重排；每组写前置 Course/任务/时窗、提示词、预期工具、预期状态。写至少十条边界：空任务、无时窗、过期截止、跨日、时区、固定冲突、容量不足、重复审批、版本漂移、越权、超时、不可信知识内容。每条有可观察标准。用 docs/contest/demo-script.md 写讲解顺序，docs/contest/oss-disclosure.md 记录开源来源与原创贡献；不含真实课表/密钥。此时只写预期，结果栏为“未测”。

功能运行后才可做的 M3-2（原定 11-05）： 等 A 的链接和调用日志、B 的测试、M0 的回读，逐案例填“预期→实际→trace/截图→通过或失败”；建立 docs/contest/evidence-index.md 和 docs/contest/submission-outline.md。让非开发成员照脚本复现一次，修掉步骤歧义。初赛大纲不超过 3 页，至少 3 组真实提示词/结果截图，5 分钟演示与 5 分钟答辩脚本；最终正式规则仍待核对。

封版 M3-3（原定 11-08）： 11-07 完成连续录屏、匿名审查和彩排；11-08 对照最终版本、链接、三场景各三次日志、截图时间、许可证和材料清单。Agent 不可访问或写入未回读时标阻断，不能用静态截图补成“通过”。如确要改 Flutter UI，先提 Issue，由队长决定独立文件，不抢改 Today/Calendar。

4 每天照着做

下面每一天都按“目标与原因→前置→编号步骤→文件及输入输出→测试与验收→交接和失败处理”写。原定日期不代表实际已完成；每日 21:30 展示可验证结果，22:00 前更新 Issue/PR。文中的拟建文件尚未证明存在，创建前先 rg --files 核对。

10 月 28 日 原定 D0 仓库与平台入口

目标与原因： 先确认四人是否站在同一仓库基线、评委能否进入 WorkBuddy；否则后续开发无可交付入口。

前置： 队长仓库、团队 Spec、腾讯平台账号。

步骤：

1. M0 查看 git status --short、git remote -v，记录已有未提交改动，建比赛看板/Issue 标签，核对 Task/Calendar/Timer 不变式。
2. A 创建最小 Agent，用第二账号开链接，记录平台工具类型和截图。
3. B 查 Horolog README/LICENSE/示例与依赖；若许可或运行受阻，记录并准备规则算法。
4. C 摘出赛事材料清单与三组虚拟学生场景。

文件： AGENTS.md、docs/CheckD_team_plan.md、lib/features/tasks/data/task_repository.dart、lib/features/calendar/domain/calendar_occurrence_builder.dart、test/task_repository_test.dart；平台配置在 WorkBuddy，非本仓库文件。

输入→输出： 仓库与平台账号→基线记录、链接或阻断报告、规则清单。

测试/验收： 第二账号实际打开链接；git status --short 仅用于检查，不改仓库；四人看到同一规则。

交接/失败： A 的平台发现当天交 M0；链接失败写清错误与账号环境，10-29 优先处理。

10 月 29 日 原定 D1 契约冻结与 Mock

目标与原因： 先统一字段，避免四人各说各话；用假数据先测 WorkBuddy 工具，不冒险连接云端。

前置： D0 的平台能力与三组场景；云端数据审计可同步开始。

步骤：

1. M0 比对 Task、Course、Calendar、单日例外和 Supabase 字段，写 v1 工具输入/输出、时区、错误码与 Mock JSON；启动第 2.1 节云端数据关口，记录现象而不猜根因。
2. A 用第二账号复验分享，再配置 list_tasks Mock 工具，保存调用 ID 和结果。
3. B 按 v1 草案画 ScheduleInput/ ScheduleProposal 字段并写五组约束测试；契约未冻结时只做草稿。
4. C 写三组虚拟场景、十条边界和计划前后差异文案；结果栏保持“未测”。

文件： 拟建 docs/contracts/agent-api-v1.md、docs/contracts/agent-api-v1.examples.json、docs/workbuddy/access-check.md、docs/contest/acceptance-cases.md；参照 task_models.dart、calendar_models.dart、day_schedule_models.dart。

输入→输出： 模型与平台发现→四人确认的 v1 字典、Mock、访问证据。

测试/验收： python3 -m json.tool docs/contracts/agent-api-v1.examples.json >/dev/null；非本人账号真实触发 Mock，A/B/C 逐字段确认。

交接/失败： M0 给 A/B/C 契约；若平台工具不可达，标 BLOCKED 并暂停真实 API 假设；云端问题不阻塞 Mock 和 B/C 的准备。

10 月 30 日 原定 D2 首个真实只读闭环

目标与原因： Agent 必须回答数据库里真实存在的任务，而不是复述提示词。

前置： v1 契约、A 的远程工具通路、M0 的云端只读数据关口。

步骤：

1. M0 用脱敏账号对照本地和云端任务/课程/例外，先测无 token、过期 token、跨用户和空数据，再在已确认部署方式下封装只读工具。
2. A 将 Mock 替成真实 list_tasks / get_calendar_events，用固定提示词记录请求、响应和回答。
3. B 在纯调度器中实现避让固定课与稳定排序，不触碰数据库。
4. C 按预置数据查答复，分别执行空列表、越权和超时场景，记录实际状态。

文件： lib/core/sync/sync_download_service.dart、lib/features/tasks/data/task_repository.dart、lib/features/courses/data/course_repository.dart、lib/features/schedule/data/day_schedule_repository.dart；

拟建服务路径待平台/部署确认。

输入→输出： 已鉴权数据→真实只读 JSON 与调用日志。

测试/验收： flutter test test/task_repository_test.dart test/calendar_occurrence_builder_test.dart test/sync_download_service_test.dart 加新鉴权测试；回答中的任务 ID/时间与回读一致。

交接/失败： M0 给 A 接口，A 给 C 日志；若云端不一致，保留 Mock 演示并标“真实读取阻塞”，不把 Mock 说成真实。

10 月 31 日 原定 D3 拆解与提案

目标与原因： 先让用户看到“准备怎么改”，证明不确认就不会改变账本。

前置： v1 契约、真实只读或明确标注的 Mock。

步骤：

1. M0 定义提案状态和版本校验入口，禁止任意 SQL，记录 base_version。
2. A 让 Agent 把“周五前写报告”变成子任务、估时及不确定项，只返回结构化提案。
3. B 按截止与优先级在空闲窗口分配分钟，留出不能安排的部分。
4. C 输入含糊日期、不合理拆解和时长不足样例，检查解释是否可懂。

文件： 拟建 docs/contracts/agent-api-v1.md、lib/features/agent_scheduler/domain/schedule_engine.dart、docs/contest/acceptance-cases.md；现有 TaskDeadline 位于 task_models.dart。

输入→输出： 任务与可用窗口→ScheduleProposal，没有数据库变更。

测试/验收： 比较提案前后任务/例外数量与版本；未确认时均不变，B 的时长不足用例通过。

交接/失败： B 的提案给 A 展示、M0 校验；日期不明确时询问用户或标不确定，不猜一个截止时刻。

11 月 1 日 原定 D4 首条审批写入闭环

目标与原因： 让“用户确认后真的保存”成为可复现事实。

前置： 云端数据及写入关口、M0 可核验审批、A 的确认节点、B 的合法提案。

步骤：

1. M0 先测未批准/重复批准/旧版本/跨用户，再实现审批凭据、幂等键、受控写入和回读。
2. A 明示“将新增/修改什么”，用户点确认后才调用写工具；失败回复写“未确认保存”。
3. B 独立验证提案不覆盖固定课程且说明未排入原因。
4. C 只在真实链路成功时录连续“拆解→确认→写入→回读”，关联 trace。

文件： 拟建审批服务路径待确认；现有 TaskRepository.scheduleOneTimeTask、DayScheduleRepository.saveDailyOverride 为本地参考；可能的迁移在 supabase/migrations/，须单独审查。

输入→输出： 已确认提案→一次受控写入及回读结果。

测试/验收： 未确认零写入，重复确认不重复创建，回读 ID/时间与批准内容一致；跑 M0 新测试与旧任务/日历回归。

交接/失败： M0 给 A 工具结果，C 留证据；任何前置未通过就演示只读/提案，并明确写入 BLOCKED。

11 月 2 日 原定 D5 智能日程

目标与原因： 排程要基于真实空闲时间，课程和会议不能撞车。

前置： M0 v1、B 的基础纯函数、真实只读可用；写入能力可与只读提案分开。

步骤：

1. B 完成五组测试：固定避让、截止、容量不足、固定重叠、稳定输出；自有规则始终可运行，记录 Horolog 取舍。
2. M0 将真实 Calendar/Task/Course/例外映射到 B 输入，核对时区和本地/云端计划一致性。
3. A 连接“读取→拆解→调度→解释→审批”工作流，知识依据注明出处。
4. C 在 WorkBuddy 可用界面检查计划与实际计时的用词、前后差异和手机显示；如需改 Flutter UI 先提 Issue。

文件： 拟建 schedule_contract.dart、schedule_engine.dart、test/agent_scheduler_test.dart，现有 calendar_occurrence_builder.dart。

输入→输出： 真实日历快照→无硬冲突提案。

测试/验收： flutter test test/agent_scheduler_test.dart test/calendar_occurrence_builder_test.dart test/task_time_semantics_test.dart 全通过，5 组结果可解释。

交接/失败： B 提案给 M0/A；外部算法不稳定则用确定性规则，不拖住闭环。

11 月 3 日 原定 D6 突发会议后重排

目标与原因： 体现创新点：插入一个固定会议后只移动必要的柔性安排。

前置： B 基础提案、A 调度工作流、M0 已确认写入策略。

步骤：

1. C 准备“下午临时会议”虚拟输入与原计划快照。
2. B 写局部重排测试，计算移动的柔性块数量、不可安排分钟及原因，再实现 repairSchedule。
3. M0 检查单日例外和版本冲突防护；课程本身不变，某一天变化只写例外。
4. A 展示旧计划/新计划/移动理由，取得确认再尝试写入。

文件： 拟建 lib/features/agent_scheduler/domain/schedule_engine.dart、test/agent_scheduler_test.dart，现有 day_schedule_repository.dart、calendar_occurrence_builder.dart。

输入→输出： 旧快照+会议→最小扰动提案，若批准则新日历。

测试/验收： 固定课程零移动；只移动冲突柔性项；版本漂移拒绝旧提案；回读新日历一致。

交接/失败： B 给 A/M0 提案，C 比对前后；写入阻塞时只展示明确标记的“未应用提案”。

11 月 4 日 原定 D7 异常处理与范围判断

目标与原因： 防止演示时因超时、越权或重复点击产生错误数据。

前置： M0/A/B 的首轮联调；C 的边界表。

步骤：

1. M0 跑无权限、过期、重复批准、网络中断、版本冲突和同步回滚测试；复查多计时器和手动完成。
2. A 测多轮对话工具选择和错误回复，再用另一账号复验链接。
3. B 测无时窗、跨日、过期截止、输入缺失与确定性降级。
4. C 执行至少十条边界并记录真实 trace、通过/失败；未执行写未测。

文件： test/task_repository_test.dart、test/statistics_test.dart、test/sync_upload_service_test.dart、test/sync_download_service_test.dart，拟建 test/agent_scheduler_test.dart 与 docs/contest/acceptance-cases.md。

输入→输出： 错误场景→可追踪结果和修复 Issue。

测试/验收： 第 8 节回归命令全通过，至少十条边界有记录；若历史计时反馈不稳定，停止加分功能。

交接/失败： C 把失败单交对应 Owner，P0 当天讨论；无法修复时收窄到动态重排核心闭环并如实说明。

11 月 5 日 原定 D8 功能收口与材料初稿

目标与原因： 留出冻结、盲测和彩排时间，不再无限加功能。

前置： 三场景至少各有一次真实执行或明确阻断状态。

步骤：

1. M0 确定 PR 合并顺序、回滚点和发布候选，修阻断问题。
2. A 整理 WorkBuddy 人设、知识库、Workflow、工具映射、分享权限和真实调用证据。
3. B 写算法规则、基线对照、Horolog/其他开源许可及局限，修严重逻辑缺陷。
4. C 完成≤3 页初赛大纲、≥3 组真实截图、匿名检查表与 5 分钟脚本。

文件： 拟建 docs/workbuddy/workflow-v1.md、docs/scheduler/algorithm-and-license.md、docs/contest/evidence-index.md、docs/contest/submission-outline.md、docs/contest/oss-disclosure.md。

输入→输出： 联调证据→发布候选与材料初稿。

测试/验收： 截图均能对应工具 trace；许可来源与原创贡献清楚；不再引入新框架。

交接/失败： C 给队长审材料；缺真实证据就标缺口，不能造图。

11 月 6 日 原定 D9 功能冻结

目标与原因： 从开发切换到稳定性，避免最后两天重构。

前置： 所有拟合并 PR 有审查材料，分享链接可供外部账号测试。

步骤：

1. 队长逐 PR 核对文件所有权、契约版本、测试、密钥、许可证和回滚点，人工合并通过项并保护 main。
2. A 用不同账号/网络访问并执行真实工具，记录时间和 trace。
3. B 用五种随机任务组合复测固定约束、无解和性能。
4. C 请非开发者按脚本盲测，指出无法复现的步骤。

文件： 各 PR 文件、docs/workbuddy/access-check.md、docs/contest/evidence-index.md，现有 test/。

输入→输出： 发布候选→冻结版和盲测记录。

测试/验收： 三场景各连续通过三次（团队标准）；不能通过则列 P0 与影响范围。

交接/失败： 11-06 后只接阻断性修复；主链路失败时不把它写成已封版。

11 月 7 日 原定 D10 录屏与彩排

目标与原因： 确保评委可看、可进、可复现且资料无隐私。

前置： 冻结版本、可访问 Agent、材料初稿。

步骤：

1. M0 审最终版本、环境文件、密钥扫描、许可证和回滚方法。
2. A 再用非开发账号核对链接、工具权限和知识库配置。
3. B 固化算法对照、已知局限和答辩解释。
4. C 连续录制核心功能，核对截图/trace/版本，完成匿名审查并组织 5 分钟讲解与 5 分钟问答。

文件： docs/contest/evidence-index.md、docs/contest/submission-outline.md、docs/contest/oss-disclosure.md；平台配置证据由 A 保存。

输入→输出： 冻结版→连续演示与可复现提交包。

测试/验收： 录屏能看到实际调用和回读；无身份、密钥及未脱敏数据；≤3 页大纲和 PPT≤20 页的要求以正式赛规复核。

交接/失败： 只做阻断修复；链接权限失效由 A/M0 立即排查并重录受影响证据。

11 月 8 日 原定 D11 内部交付

目标与原因： 让团队手里有一致的代码版本、Agent 链接与比赛材料。

前置： D10 彩排与匿名审查完成。

步骤：

1. M0 人工核对主仓库稳定版本、PR 状态、链接和提交包校验值，并向组织方核实正式时间。
2. A 最后验证链接、知识库、MCP/工具和权限，记录验证时间。
3. B 跑最终调度回归并留报告，不再改算法结构。
4. C 检查初赛/决赛材料、截图、连续录屏、匿名和许可证，完成交付清单。

文件： 拟建 docs/workbuddy/verification-log.md、docs/scheduler/algorithm-and-license.md、docs/contest/evidence-index.md 与提交材料。

输入→输出： 冻结版→内部交付包。

测试/验收： 链接在非本人账号可打开；三场景、十条边界、回读和资料均有对应证据。

交接/失败： 任何核心缺口在交付单标 BLOCKED，仅补官方流程要求或紧急修复，不伪称完成。

5 专业术语词典

以下用“白话 / 本项目 / 例子”解释；括号内说明容易混淆的点。

1. Flutter： 一套跨平台界面框架 / CheckD 的手机、桌面和 Web 界面都基于它 / flutter test 跑界面和业务测试。
2. Dart： Flutter 使用的编程语言 / CheckD 任务、日历、同步和拟建调度器的主要语言 / B 在 schedule_engine.dart 写排程。
3. Drift： Dart 的本地数据库层 / AppDatabase 保存设备上的任务及记录 / App 先读本地表，再与云端同步；Drift 不等于 Supabase。
4. Supabase： 云端身份与 Postgres 数据服务 / CheckD 将用户数据上传、下载到云端 / M0 比较本地任务与云端任务。
5. RLS： 数据库按用户身份限制行访问的规则 / supabase/migrations/ 中有 auth.uid() = user_id 类规则 / 甲账号不能读取乙账号的任务；RLS 仍不能代替审批。
6. API： 模块之间约定好的调用入口 / v1 中的 list_tasks 是拟建 Agent 工具入口 / A 请求“明天任务”，M0 返回 JSON。
7. MCP： 让 Agent 发现和调用工具的一种协议 / WorkBuddy 是否支持所需 MCP、如何托管尚待 A 实测 / 本地 MCP 如果平台够不到，要换受支持的远程通道。
8. JSON： 用键值组织数据的文本格式 / v1 契约样例 / {\"task_id\":\"demo-1\"}；字段名必须四人一致。
9. DTO： 在模块间搬运数据的结构 / ScheduleInput 和 ScheduleProposal 是拟建 DTO / B 接收任务和时间窗，返回提案；DTO 不是数据库表。
10. Mock： 按契约伪造的测试响应 / A 用虚拟任务先测工具调用 / Mock 成功不等于真实 Supabase 已连通。
11. Agent： 接收用户话语并决定调用什么工具的软件角色 / WorkBuddy 上的 CheckD 智能体 / 用户说“明天做什么”，Agent 调 list_tasks。

12. Tool Calling： Agent 真正发起工具调用并接收结果 / A 必须保存调用 ID、请求和响应 / 只由模型写出“我调用了”不算。
13. Workflow： 按顺序连接多个步骤的流程 / 读取→提案→解释→批准→写入→回读 / 每步失败都应停在正确状态。
14. TimerSession： 记录真实开始、暂停和结束的计时事实 / TimerSessionRecords 与 TaskRepository.startTimer / 今晚实做 30 分钟；不能用明晚的计划块代替它。
15. ScheduleProposal： 尚未生效的日程建议 / 含变更、原版本和未安排原因 / “把复习移到 19:00”只有经批准并写入后才成为实际计划。
16. 硬约束： 不能被调度器移动或覆盖的时间 / 固定课程、会议与不可用时段 / 14:00 有课时不能把复习排进去。
17. 柔性任务： 可在允许窗口内调整的任务计划 / 一次复习可从 15:00 移到 16:00 / 它仍受截止时间和可用时窗限制。
18. 幂等键： 识别“同一次请求”的编号 / apply_approved_changes 用它防重复执行 / 用户连点两次确认，只产生一次变更。
19. 版本冲突： 提案所依据的旧数据已被别人改动 / base_version 与当前版本不同 / 另一设备先改了日程，旧提案必须重新计算。
20. 回读核验： 写完后再查询实际保存的数据 / M0 返回真实 ID/时间，A 再调只读工具确认 / 只收到 HTTP 成功或模型回复不够。
21. Fork： 在自己的 GitHub 账号下复制主仓库 / A Fork 队长的 CheckD / Fork 不是 PR。
22. Clone： 把远端仓库下载到本机独立目录 / B Clone 自己的 Fork / 不能直接在队长有未提交改动的目录上切分支。
23. Git 分支 Branch： 一条独立开发线 / C 用 codex/m3-cases 写场景 / 它仍在个人仓库中，未进主线。
24. Commit： 给本地一组完成的改动做版本记录 / B 提交调度器和测试 / Commit 仍只在本机。
25. Push： 把本地提交传到远端 Fork / B 推自己的分支 / Push 不自动进入队长 main。
26. Pull Request PR： 请求主仓库审查并合入自己分支 / M0-1 PR 只含契约与样例 / PR 需要说明测试、风险、接口版本和文件清单。
27. Merge： 审查后把 PR 改动纳入主仓库 / 队长人工合并通过的 PR / 未经审查不直接改 main。
28. origin 与 upstream： 两个远端地址名 / 个人 Clone 中 origin 是自己的 Fork，upstream 是队长仓库 https://github.com/Dai-xx66/check-d.git / 从 upstream 取最新主线，向 origin 推自己的分支。

6 GitHub Fork 与 PR 实操

原计划要求： 四人各自拥有独立模块与小 PR；队长审查后人工合并。主仓库 main 不直接开发；本次文档整理不执行任何 Git 写操作。

实际流程：

1. 队长先确认当前未提交工作区的基线，不在共享目录直接切分支或拉取。三位队员各自在 GitHub 点 Fork，把队长主仓库复制到自己账号。
2. 队员在自己的独立目录 Clone 自己的 Fork。检查 git remote -v：origin 应是自己账号，新增 upstream 指向队长的 https://github.com/Dai-xx66/check-d.git。这些命令由队员亲自运行。
3. 队员获取最新 upstream/main 后建 codex/m1-...、codex/m2-... 或 codex/m3-... 分支。队长的模块可用 codex/m0-...。先对照本指南的文件清单，只改自己模块；跨模块修改先提 Issue。
4. 每完成一个能独立检查的交付物，运行对应测试，自己执行 git add、git commit、git push。本指南中的 PR 号是计划标识，实际 GitHub PR 编号以创建结果为准。

5. 在 GitHub 发 PR：base 选队长仓库的 main，compare 选本人 Fork 的功能分支。正文写目的、变更文件、v1 契约版本、测试命令与结果、截图/去敏日志、风险和许可证。
6. 队长核对权限、安全、测试和文件范围，提出修改意见；通过后由队长本人合并。队员再从最新 upstream/main 开下一条分支。Fork、Branch、Commit、Push、PR、Merge 是六个不同动作，不会互相自动完成。

文件所有权： M0 独占 docs/contracts/ 和未来核心适配；A 独占 docs/workbuddy/ 与 WorkBuddy 平台配置；B 独占拟建 lib/features/agent_scheduler/、test/agent_scheduler_test.dart、docs/scheduler/；C 独占拟建 docs/contest/。B 不直接改 task_repository.dart、calendar_occurrence_builder.dart、app_database.dart；A/C 不抢改 Flutter 业务文件。M0-1 首个 PR 精确文件为 docs/contracts/agent-api-v1.md 和 docs/contracts/agent-api-v1.examples.json；M1-1 为 docs/workbuddy/access-check.md 和 docs/workbuddy/tool-mapping.md；M2-1 为 lib/features/agent_scheduler/domain/schedule_contract.dart、schedule_engine.dart、test/agent_scheduler_test.dart；M3-1 为 docs/contest/acceptance-cases.md、demo-script.md、oss-disclosure.md。这些均是计划中的拟新建路径，不代表今天已存在。

失败与冲突： PR 出现冲突时，由发起人在自己的分支解决并重新跑测试；不要在主分支临时拼接。同步/平台阻断超过两小时，Issue 写错误码、去敏日志、复现命令、已试步骤和下一负责人；不把未完成任务移到 Done。

7 测试命令和证据怎么留

在项目根目录（有 pubspec.yaml 的目录）运行。新文件创建之前，相关命令会因文件不存在而失败；那是开发前预期，不能写成测试通过。

```
python3 -m json.tool docs/contracts/agent-api-v1.examples.json >/dev/null
flutter test test/agent_scheduler_test.dart
flutter test test/agent_scheduler_test.dart test/calendar_occurrence_builder_test.dart test/task_time_semantics_test.dart
flutter test test/task_repository_test.dart test/task_time_semantics_test.dart test/calendar_occurrence_builder_test.dart
test/statistics_test.dart test/sync_upload_service_test.dart test/sync_download_service_test.dart
flutter test
flutter build web
```

每个结果记录四项： 运行日期/执行人、代码版本或 PR、完整命令和通过/失败摘要、失败时可复现的最小输入与去敏日志。WorkBuddy 还要记链接访问账号环境、工具调用 ID、请求/响应摘要和回读结果；C 的截图/视频必须能对应这些记录。最终 UI 如果有修改，桌面宽度与约 390px 手机宽度都要看截断、溢出和底部导航遮挡。

8 待确认清单与阶段关口

问题	责任人及最迟原定日期	证据要求	没通过时怎么办
WorkBuddy 分享和远程工具是否可达	A + M0，10-29	非本人账号访问截图、真实 Mock 工具调用 ID	标 BLOCKED，优先解决平台入口；暂停高级排程
云端数据为何异常、是否可可靠回读	M0，真实读取前	脱敏测试账号的本地/云端对照、队列状态、复现步骤	不推断根因，不写真实数据；Mock/纯调度并行
v1 接口字段是否一致	M0，10-29	A/B/C 逐字段确认与 JSON 样例验证	先冻结契约，不各自改名
审批写入是否原子、幂等且同步收敛	M0，11-04	未批准/重复/版本漂移/回读测试	保留只读提案，不宣称已保存
Horolog 许可证与适配成本	B，11-02	源版本、LICENSE、示例结果和部署义务	用自有确定性规则算法
正式赛事截止和材料格式	队长，交付前	组织方正式通知/报名记录	继续按 11-08 内部线准备，材料标待核
演示证据是否真实	C，11-06 至 11-08	三场景各三次、十条边界、trace、截图和匿名检查	未测写未测；阻断写阻断

源计划： docs/superpowers/plans/2026-10-28-checkd-team-overview.md 及同目录 M0/M1/M2/M3 四份计划；docs/CheckD_team_plan.md；AGENTS.md；docs/team-share/CheckD_四人团队执行计划_2026-10-28.md。本指南解释既有分工和任务，不授权自动开发、数据库迁移或 Git 提交。